



Los medios en Cloudflare R2

Crear el bucket, el token y las variables; comprobar el ciclo completo con ``pnpm r2:verificar`` antes de que entre nadie; y, si ya había medios en el disco, moverlos sin perderlos. Lo que no se ha ejecutado está marcado.

Por qué R2 y no el disco

Cada visita a una foto **vuelve a leer el original** para incrustarle su marca de agua. Ese diseño se paga dos veces con un proveedor que cobre el tráfico de salida; R2 no lo cobra.

Y hay una razón que no es de dinero: con `STORAGE_DRIVER=fs` **los medios no entran en ninguna copia de seguridad** —`scripts/backup.sh` vuelca la base, no esos bytes—. Si se pierde el disco del VPS, se pierden las fotos de un cliente.

Aviso honesto, y sigue en pie. El adaptador de R2 firma SigV4 a mano y sus pruebas usan un `fetch` de mentira: comprueban qué petición **sale**, no que alguien la acepte. Se ha verificado contra MinIO, que valida igual, y con el script de este documento. **R2 de verdad no lo ha visto nadie todavía.** Por eso el estreno se hace con el paso 4 delante y **sin clientes dentro.**

1. El bucket

En el panel de Cloudflare, R2 → **Create bucket**:

Ajuste	Valor	Por qué
Nombre	<code>vistta-medios</code>	Es el que va en <code>R2_BUCKET</code> ; distingue mayúsculas
Acceso público	Desactivado	Ver el aviso de abajo. No actives el dominio <code>r2.dev</code>
Ubicación	Europa	Residencia de datos coherente con el VPS (Alemania)

No actives el dominio público del bucket, ni ahora ni después. Los medios se sirven **solo** por la API, que exige tres cosas —firma válida, fila en `pass_media` y `status='ready'`— y que le mete a cada imagen la marca de agua de esa visita. Un dominio público sirve **el original limpio** a quien tenga la clave, y las claves son adivinables por diseño (`u/<perfil>/<uuid>`). Sería regalar exactamente lo que este producto cobra.

Ubicación UE: hay dos cosas distintas y solo una garantiza

Cloudflare ofrece dos ajustes que suenan igual y no lo son:

- **Location hint** (`weur`, `eeur`...): una *sugerencia* de dónde colocar el bucket. El endpoint no cambia. Es una preferencia, no una garantía.
- **Jurisdicción** `eu`: un compromiso de que los datos **no salen de la Unión Europea**. Es lo que se puede escribir en `legal/rat.md` sin mentir. **Se elige al crear el bucket y no se puede cambiar después.**

Si eliges jurisdicción `eu`, el endpoint deja de ser `<cuenta>.r2.cloudflarestorage.com` y pasa a ser `<cuenta>.eu.r2.cloudflarestorage.com`, y entonces **hay que ponerlo en el `.env`**:

```
R2_ENDPOINT=https://<R2_ACCOUNT_ID>.eu.r2.cloudflarestorage.com
```

Sin esa variable, la API busca el bucket donde no está y el error que verás es un `NoSuchBucket` que señala al sitio equivocado. Con jurisdicción normal, `R2_ENDPOINT` se deja sin poner.

No ejecutado: el nombre exacto del ajuste en el panel y la forma del endpoint con jurisdicción no se han visto en una cuenta real. Confírmalos en el panel; el script del paso 4 te lo dirá enseguida si no coinciden.

2. El token

R2 → Manage API tokens → Create API token:

Ajuste	Valor
Permisos	Object Read & Write
Alcance	Solo <code>vistta-medios</code> , no toda la cuenta
Caducidad	Sin caducar, o anótala en el calendario

Object Read & Write, no solo lectura, y esto no es una preferencia: sin permiso de borrado, ni la purga ni el reaper de huérfanos pueden limpiar nada. El bucket crece para siempre y la factura con él. El script del paso 4 detecta justo ese caso.

Cloudflare enseña la clave secreta **una sola vez**. Si la pierdes, se genera otra; no se recupera.

3. Las variables

En el `.env` del servidor, junto a `compose.prod.yml`:

```
STORAGE_DRIVER=r2
R2_ACCOUNT_ID=
R2_ACCESS_KEY_ID=
R2_SECRET_ACCESS_KEY=
R2_BUCKET=vistta-medios
# Solo con jurisdicción `eu`; en otro caso, se deja fuera.
R2_ENDPOINT=
```

`MEDIA_SIGNING_KEY` NO SE CAMBIA NUNCA

No tiene nada que ver con R2 y por eso se dice aquí, que es donde la gente abre el `.env` y toca cosas. Esa clave firma **todas** las URL de medios. Cambiarla invalida de golpe:

- las URL de cualquier pase **abierto en ese momento**, delante del cliente que lo está mirando;
- las de todos los pases enviados y aún no abiertos.

No se puede deshacer regenerándola: la firma vieja ya no vale. Si algún día hay que rotarla, es una operación planificada con todos los pases cerrados, no un cambio de `.env`.

4. Comprobar ANTES de que entre nadie

```
pnpm r2:verificar
```

Y en el servidor, que es donde vive el `.env` con las credenciales, desde la propia imagen:

```
docker compose -f compose.prod.yml run --rm api node dist/verificar-r2.js
```

Hace el ciclo entero contra el bucket real —**subir, leer, comprobar que los bytes son idénticos, comprobar que una clave inexistente devuelve `null`, borrar, y comprobar que ya no está**— con una clave suya bajo `verificacion/`, que borra también si algo falla. No toca ningún medio.

Cuando va bien:

- ✓ subir
- ✓ leer (256 bytes idénticos, tipo `application/octet-stream`)
- ✓ una clave que no existe devuelve `null`
- ✓ borrar
- ✓ ya no está

R2 responde al ciclo completo: subir, leer, borrar.

Y cuando no, dice cuál de las cuatro variables mirar. Estos cinco casos están **probados contra MinIO**, provocándolos a propósito:

Lo que verás	Lo que pasa de verdad
<code>403 SignatureDoesNotMatch</code> al subir	<code>R2_SECRET_ACCESS_KEY</code> no corresponde, o el reloj va desviado
<code>403 InvalidAccessKeyId</code>	<code>R2_ACCESS_KEY_ID</code> no existe en esa cuenta
<code>404 NoSuchBucket</code>	El nombre del bucket, o el endpoint (¿jurisdicción <code>eu</code> ?)
Sube y lee, falla al borrar	El token es de solo lectura. Es el caso caro: nada podrá limpiarse
faltan <code>R2_...</code>	Una variable vacía en el <code>.env</code>

Las credenciales **no se imprimen** en ningún caso.

5. Si ya había medios en el disco (`fs` → R2)

Solo aplica si estrenaste con `STORAGE_DRIVER=fs`. Si empiezas directamente en R2, sáltate este apartado.

La regla que decide si sale bien: las claves de los objetos en R2 tienen que coincidir, carácter a carácter, con la columna `vistta.media.storage_key` de la base. La fila es la que manda; si un objeto no está exactamente en su clave, ese medio no se encuentra y el pase lo enseñará roto.

Hoy esas claves tienen la forma `u/<perfil>/<uuid>`, **sin extensión**. Cualquier herramienta que "arregle" nombres o añada sufijos rompe la correspondencia.

Lo que hay que excluir, y que no es evidente

El adaptador de disco escribe **dos ficheros por medio**: el objeto y, al lado, un `<clave>.tipo` con el tipo MIME. **Esos `.tipo` no deben subirse**. Si suben, quedan en el bucket como objetos que ninguna fila referencia, así que **ni la purga ni el reaper los borrarán jamás**: los pagarás para siempre.

```
# 1. Sacar los medios del volumen de Docker al disco del servidor.
docker compose -f compose.prod.yml cp api:/medios ./medios-copia

# 2. Configurar rclone con las mismas credenciales (una vez).
rclone config create r2 s3 provider=Cloudflare \
  access_key_id=<R2_ACCESS_KEY_ID> secret_access_key=<R2_SECRET_ACCESS_KEY> \
  endpoint=https://<R2_ACCOUNT_ID>.r2.cloudflarestorage.com

# 3. Copiar, DEJANDO FUERA los .tipo.
rclone copy ./medios-copia r2:vistta-medios --exclude "*.tipo" --progress
```

Con `aws s3 sync` el equivalente es `--exclude "*.tipo"` y `--endpoint-url https://<R2_ACCOUNT_ID>.r2.cloudflarestorage.com`.

Cotejar antes de cambiar el `.env`

Cuenta lo que hay a cada lado. Tienen que dar el mismo número:

```
# Objetos en el bucket
rclone size r2:vistta-medios

# Medios que la base espera encontrar
docker compose -f compose.prod.yml exec db \
  psql -U vistta -d vistta -c \
    "SELECT count(*) FROM vistta.media WHERE status = 'ready';"
```

Si el bucket tiene **más** objetos que filas, probablemente se colaron los `.tipo`. Si tiene **menos**, falta contenido y algún pase se verá roto.

El tipo MIME de los objetos da igual

`rclone` adivinará un `Content-Type` al subir, y no importa: al servir, el tipo sale de la columna `media.mime`, y las imágenes salen del reencodificado de Sharp con su marca. El tipo guardado en el objeto no se usa en ninguna de las dos rutas que sirven medios. (Comprobado leyendo `src/routes/media.ts` y `src/routes/profiles.ts`.)

Y después

```
# Cambiar STORAGE_DRIVER=fs por r2 en el .env, y:
docker compose -f compose.prod.yml up -d
```

No borres `./medios-copia` ni el volumen `medios` hasta haber abierto un pase de verdad y visto las fotos. Es la única red que te queda si algo no cuadró.

6. La comprobación que ninguna máquina hace

Con R2 puesto, entra al panel, **sube una foto, genera un pase, ábrelo y mira que la foto lleva la marca de agua encima**. El script del paso 4 demuestra que R2 responde; esto demuestra que el producto funciona. Son cosas distintas.

Y el invariante, como siempre (`docs/11` §7): abrir el pase dos veces da **200 y luego 410**.

Qué se ha ensayado y qué no

Probado de verdad	<code>scripts/verificar-r2.ts</code> contra MinIO : el ciclo completo en verde, y los cinco fallos de la tabla del paso 4 provocados a propósito, cada uno con su diagnóstico y sin filtrar credenciales
Leído, no ejecutado	Que el tipo MIME del objeto no se usa al servir: sale de <code>media.mime</code> en las dos rutas
Nunca ejecutado	R2 de verdad : ni el bucket, ni el token, ni el endpoint de jurisdicción <code>eu</code> , ni la copia con <code>rclone</code> . Nada de este documento ha tocado Cloudflare

El paso 4 existe precisamente para que esa última fila deje de dar miedo: es una orden, tarda diez segundos y dice qué pasa.